I don't use this form very much, though I think it would be helpful if you had additional creation logic on a new test object.

This is the strategy by which I determine which of these methods to use:

- Use attribute_for only in the specialized case of needing a valid set of hash attributes. Again, in my experience that's most likely in a controller test.
- Use create only if the object absolutely must be in the database. Typically, this is because the test code must be able to access it via an ActiveRecord finder. Sometimes this is because the objects are in an ActiveRecord relationship and it is much easier to specify that with create. However, create is much slower than any of the other methods, so it's also worth thinking about whether there's a way to structure the code so that persistent data is not needed for the test.
- Use build in some cases where you don't actually need items in the database but Rails wants to force things to be saved (like associations). Factory_bot gets snippy about stubbed objects not being able to access the database, and this can cause problems testing objects with associations. Sometimes, though, it's better to rewrite the spec to add the association in the setup, as you did in the recent examples.
- In all other cases, use build_stubbed, which does everything build does, plus more. Because a build_stubbed object has a Rails ID, you can build up real Rails belongs_to associations and still not have to take the speed hit of saving to the database.

Prescription 17	Your go-to build strategy for factory_bot should be build_stubbed unless there is a need for the object to be in the database during the test.

In case you need to create a set of objects together, all the build strategies have two special forms: *_pair and *_list. The pair methods, like create_pair(:project) and build_stubbed_pair(:project), create exactly two objects of the given factory. You can still pass key/value pairs to the method, in which case the attribute overrides are applied to both objects. The list methods create an arbitrary number of items, denoted by an integer argument after the name of the factory, as in create_list(:project, 5), which creates five projects. As with the pair methods, key/value pairs can be passed in and are applied to the entire list. As far as I can tell, there's no way to specify in the call to the *_pair or *list method that the attributes should be different in different instances of the object. The same key/value pairs are applied to each instance.